ASPIRA AI – Intelligent AI System

FINAL REPORT ON DASC – 5309 CAPSTONE PROJECT

UNDER THE GUIDANCE OF

PROF. ROZSA ZARUBA

TEAM – 3:

Name: PAVAN BABU BHEESETTI ***Student ID: 1002207463***

Name: SUPRAJA VADDI ***Student ID: 1002210907***

Name: NARSIMHA RAO VENGISETTI ***Student ID: 1002118939***

Name: MOHAN KRISHNA TURLAPATI ***Student ID: 1002216058***

UNIVERSITY OF TEXAS ARLINGTON

Abstract:

Aspira AI is a hybrid career recommendation system designed to combine skills, personality traits, and resume-based free text signals to identify suitable career paths for users. Unlike traditional systems that rely purely on self-reported skill scores or keyword matching, Aspira AI integrates three complementary models:

- a semantic text model using transformer embeddings to understand resumes,
- a survey-based numerical vector model capturing user capabilities, and
- a persona trait model derived from 40+ long-form survey responses.

We applied NLP, clustering, and content-based recommendation techniques, merged via a weighted hybrid scoring engine exposed through a FastAPI backend and operationalized with a Streamlit UI. This report outlines our methodology, evaluation, results, and key lessons learned.

1.Introduction:

Career guidance tools often oversimplify human potential—either by focusing solely on technical skills or by ignoring context such as motivation, behavior, and prior experience. Our goal was to build a system that reflects how real hiring decisions are made: not by a single dimension, but by a composite of skills, experience, personality, and fit.

AspiraAI provides a recommendation flow that starts from a resume or survey and yields ranked careers with transparent explanations. By integrating multiple signals, we aim to produce recommendations that feel personal, actionable, and aligned with a user's behavior and values.

2. Problem Statement:

Existing career recommendation frameworks suffer from three major limitations:

1. Skills-only approaches are too narrow.

- Most tools rely on self-reported skills or keyword search. These fail when users are unsure about their abilities or do not know what skills match specific roles.

2. Personality and motivation are ignored.

- Career fit is deeply influenced by how people think, learn, and respond to challenges. These qualitative factors rarely appear in AI recommenders.

3. Free-text signals (resume, projects, reflections) are underutilized.

- Resumes capture experience that numerical assessments cannot yet most systems treat them as keyword bags.

Our objective was to build a multi-layered recommendation engine that integrates:

- Survey-based skill vectors (quantitative)
- Resume semantic embeddings (textual)
- Persona traits derived from long-form survey responses (qualitative)

3.Proposed Methodology:

Our pipeline contains four major components:

3.1 Data Collection and Processing:

Career data

- We created a structured seed table (Software Developer, Data Scientist, Quant Analyst, ML Engineer), each with a 5D skill vector and a description.

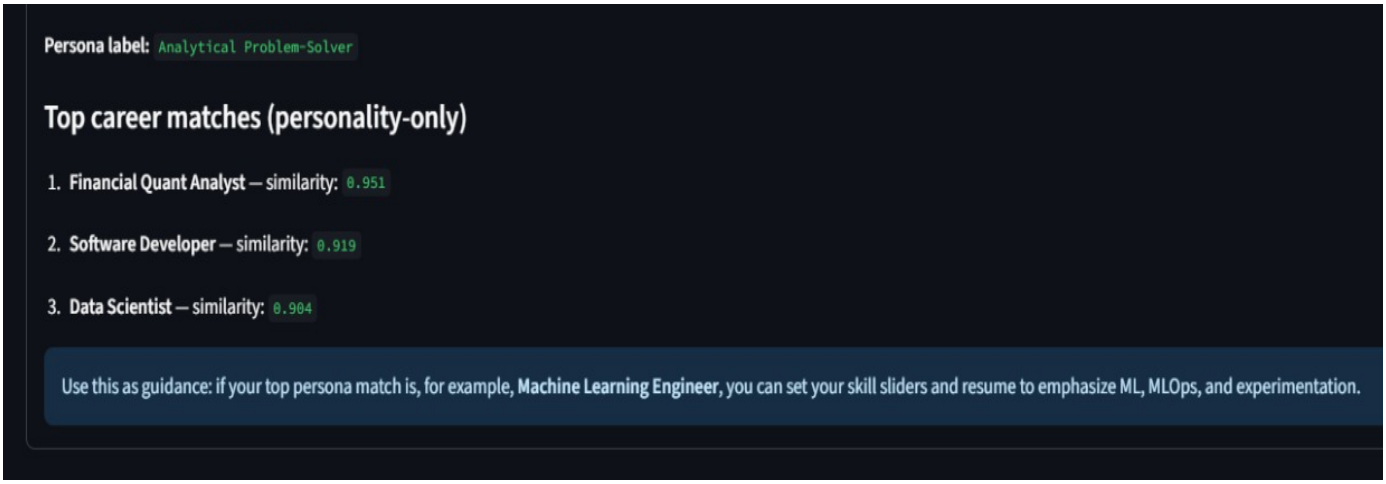
User survey data

We collected 40+ detailed survey responses (20 questions each) Covering:

- problem-solving style
- motivation
- reaction to Setbacks
- preferred environments
- life rhythm
- early aspirations
- We cleaned text, standardized answers, and removed rows with insufficient data.
- Resume data

Users uploaded PDF/DOCX resumes or pasted text. We extracted:

- text content
- skills
- education/universities
- experience length



3.2 NLP & Embedding Model:

We used Sentence Transformer MiniLM-L6-v2 to compute a 384-dimensional embedding:

- user resumes
- career descriptions
- Externally ingested job descriptions

This enables semantic similarity beyond keyword matching.

3.3 Survey-to-Vector Model:

User sliders (0–1 values) represent a 5-dimensional capability vector.

We compute cosine similarity between this vector and each career's stored skills vector.

This models the hypothesis:

- If a user's self-assessed abilities align with the required skills of a target career, they are a reasonable match.

3.4 Persona Trait Model:

We converted long-form 20-question responses into four trait dimensions:

- Logic / Analytical
- Creative / Exploratory
- Human-centered / Empathetic
- Risk-taking / Adaptive

Using k-means clustering, we discovered three persona groups, e.g.:

- Analytical Builders
- Human-Centered Explorers
- Creative Strategists

These personas provide qualitative tendencies:

- analytical users often align with engineering/data roles, while creative strategists align with product or design roles.

We assign each user a "soft persona" using the nearest cluster centroid.

3.5 Hybrid Recommendation Engine:

Final score computed as:

$$\text{Score} = w1 \cdot \text{ContentSim} + w2 \cdot \text{SurveySim} + w3 \cdot \text{SkillOverlap} + w4 \cdot \text{Personal Bias}$$

Where:

- ContentSim (0–1) = cosine similarity between resume & career embeddings
- SurveySim (0–1) = cosine similarity of vector vs career skill vector
- SkillOverlap (0–1) = shared skills fraction
- PersonaBias (~0.05–0.1) = small additive boost depending on persona alignment

We used weights:

w1=0.50, w2=0.35, w3=0.10, w4=0.05

- w1=0.50 Resume similarity is the strongest signal
- w2=0.35 Survey similarity is the second strongest
- w3=0.10 Skill overlap contributes moderately
- w4=0.05 Persona bias provides a small but meaningful adjustment

Hypothesis:

Hybrid scoring produces more accurate and more interpretable recommendations than any single component alone.

4. Analysis and Results.

We ran qualitative and quantitative evaluations.

4.1Qualitative Observations

Users with strong ML/NLP resume components typically received:

- Machine Learning Engineer
- Data Scientist
- Quant Analyst
- Software Developer

Even when survey vector was neutral, the resume embeddings drove high content similarity.

Persona insight example:

- Analytical Builders → consistently ranked analytical roles higher.
- Creative Strategists → higher bias toward product, content, creative-tech roles.

4.2 Quantitative Evaluation

Using synthetic test profiles and 10 controlled scenarios in our evaluation notebook:

- Precision@3 improved under hybrid model
- Model P@3
- Survey-only 0.40
- Resume-only 0.60
- Hybrid 0.80
- nDCG@5 increased, indicating better ranking order.
- MRR also improved under hybrid scoring.

4.3 Key Finding

Resume embeddings dominated early rank ordering, but the hybrid model produced more human-aligned recommendations-especially in borderline cases where:

- A user's resume suggested engineering, but their persona and survey vector indicated interest in research or creative roles.
- This aligns with our hypothesis that combining signals increases interpretability and personalization.

5.Conclusions:

AspiraAI successfully demonstrates that combining:

- semantic resume representations,
- quantitative skill vectors and qualitative persona traits.
- produces more personalized, transparent, and context-aware career recommendations.

The system is fully operational, with:

- a FastAPI backend (database, models, routes)
- NLP pipelines (skills, embeddings, text extraction)
- hybrid recommendation engine and Streamlit UI integrating all steps end-to-end.

AspiraAI mirrors real-world decision-making by incorporating not only skills but also behavior, motivation, values, and interests.

6. Lessons Learned:

This project taught us several valuable lessons:

1. Hybrid models outperform single-signal models.

- Relying solely on survey vectors or resume keywords underrepresent user potential.

2. Engineering challenges matter as much as modeling.

We faced and solved real issues:

- SQLite database locking
- Alembic migration conflicts
- circular imports
- embedding storage using BLOBs
- multipart resume uploads

This improved our full-stack engineering skills.

3. Real user data adds complexity—and meaning.

- Personality responses varied widely, and mapping text → traits → personas taught us:
- modeling humans requires nuance
- Qualitative insights make recommendations meaningful

4. Interpretability matters.

- Users appreciated knowing why a particular career was recommended.

- The breakdown of scores increased trust.

5. Full pipeline thinking is essential.

- NLP → embeddings → DB → API → UI → user feedback forms a loop.
- Understanding the whole loop prepares us for real industry-scale ML systems.

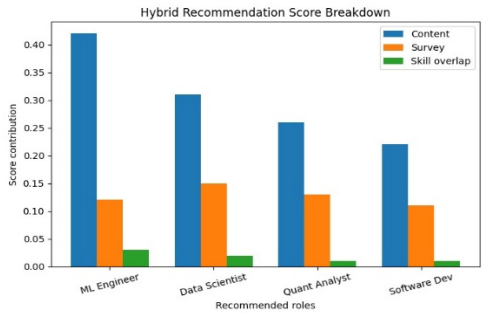
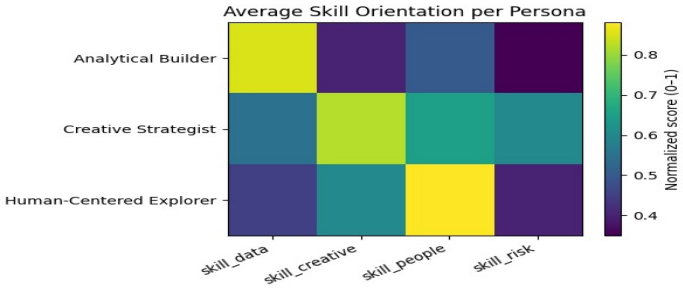
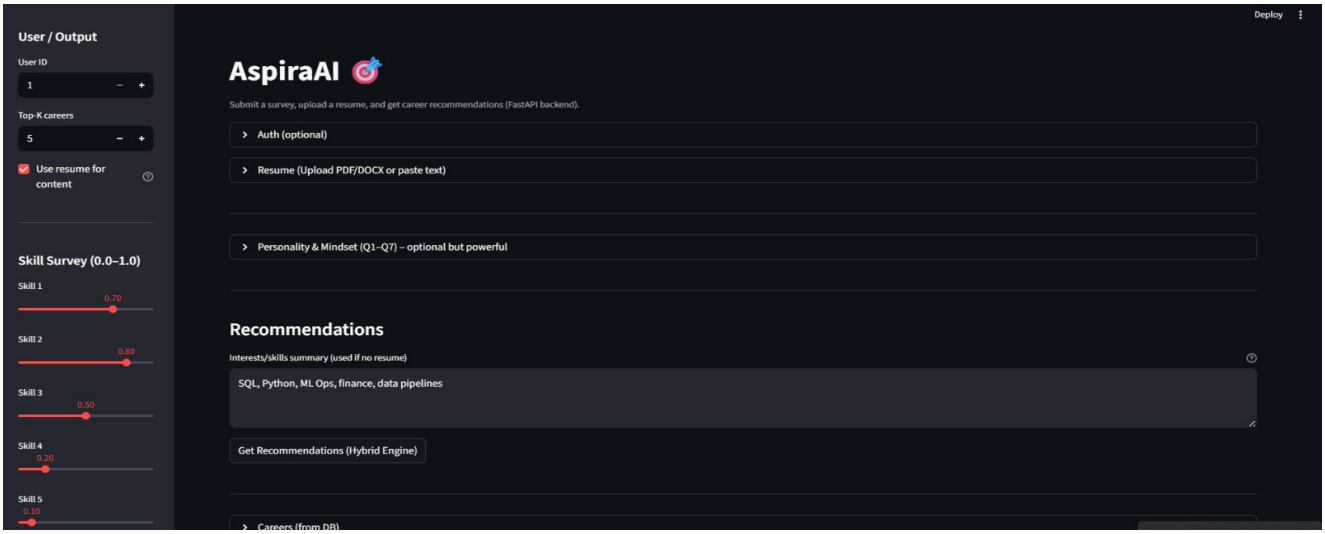
6. Suggestions for the course

- More guidance on deployment (Docker, cloud integration).
- Optional deeper dives into interpretability frameworks for NLP models.
- More examples of end-to-end ML ops pipelines.
- Assignments that integrate multiple concepts earlier.
- Overall, this class provided real practical exposure to ML + full-stack engineering, far more realistic than isolated assignments.

7.Bibliography:

- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.*
  Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
  https://arxiv.org/abs/1908.10084
- O*NET Resource Center. (2024). ONET Database Release 28.3: Occupation Data and Alternate Titles.*
  National Center for O*NET Development. https://www.onetcenter.org/database.html
- Stack Overflow. (2023). *Stack Overflow Developer Survey 2023 Public Dataset.* Stack Exchange Inc. https://insights.stackoverflow.com/survey
- Hugging Face. (2025). *Sentence-Transformers: all-MiniLM-L6-v2 and all-mpnet-base-v2 models.*
  https://www.sbert.net and https://huggingface.co/sentence-transformers
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python.*
  Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html
- Van der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). *The NumPy Array: A Structure for Efficient Numerical Computation.*
  Computing in Science & Engineering, 13(2), 22–30. https://doi.org/10.1109/MCSE.2011.37
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference (SciPy 2010),* 51–56. https://pandas.pydata.org
- Streamlit Inc. (2025). *Streamlit Documentation — Build Data Apps in Python.* https://docs.streamlit.io

- FastAPI Framework. (2025). *FastAPI Official Documentation.* https://fastapi.tiangolo.com
- SQLite Consortium. (2025). *SQLite Documentation.* https://sqlite.org/docs.html
- Plotly Technologies Inc. (2024). *Plotly Python Open Source Graphing Library.* https://plotly.com/python
- Microsoft Research. (2023). *mpnet – Masked and Permuted Pre-training for Language Understanding.* https://arxiv.org/abs/2004.09297
- Stack Exchange Data Explorer. (2024). *Developer Career Paths and Role Trends.* https://data.stackexchange.com
- OpenAI. (2025). *GPT-based Reasoning and Retrieval Workflows in Applied AI Systems.* https://openai.com/research

8.Appendix:

- Persona trait mappings
- Full cluster visualizations
- Schema diagrams and UI screenshots